

REPLICANT LAB · FICHA TÉCNICA DERIVADA DE MKDOCS

PULA

Arquitectura, despliegue, operación, seguridad y evidencia auditada.

Fuente

aplicaciones/pula.md

Versión reproducible

sha256:462d1f8361447caa5ff0ffd28263c22c7b5d818168eac32960b42189fececc68

PULA

Auditoría técnica y manual funcional de `ratienza/Pula`. **PULA es exclusivamente una POC privada para localizar y gestionar apartamentos de la web oficial SCPU en Pula, Croacia.** Su finalidad actual es ayudar en una búsqueda familiar concreta; no es ErasmusHomes ni una plataforma paneuropea.

Estado comprobado

Elemento	Evidencia
Repositorio	<code>ratienza/Pula</code> · privado
<code>main</code> auditado	<code>ff8165cf7c9d1d4c5ae670c56486c25900a5754b</code>
Alcance funcional	Búsqueda SCPU, gestión de candidatos, contacto y seguimiento Gmail
Checkout Nexus	<code>/opt/apps/Pula</code> · limpio y sincronizado; copia de consulta, no runtime validado
Producción	Cloud Run declarado por el proyecto; no inspeccionado ni modificado
Pruebas locales	TypeScript y build Vite/esbuild correctos sobre copia temporal
Tests automatizados	No existe suite; solo scripts exploratorios de scraping/Firebase
Datos versionados	<code>data.json</code> : 29 registros <code>new</code> con 29 direcciones de contacto

Calidad general: **5/10**. La POC compila y concentra el flujo funcional para Pula, pero carece de tests, control de acceso, persistencia robusta y trazabilidad reproducible de Cloud Run.

Separación obligatoria de proyectos

`ratienza/ErasmusHomes` es otro repositorio y otro proyecto. Contiene el concepto comercial paneuropeo, el ejemplo visual de Bolonia/UNIBO, Smart Deposit Audit y el dossier ReportLab. Actualmente debe tratarse como **concepto no validado**, no como funcionalidad implementada de PULA.

La etiqueta `v0.1-poc-beta` y los artefactos ErasmusHomes presentes en el historial de `ratienza/Pula` son contaminación histórica y deuda de higiene del repositorio. No definen el alcance de PULA y no se usan como fuente funcional en esta ficha. Cualquier limpieza futura de esa historia o etiqueta requiere un encargo específico sobre el repositorio PULA.

Arquitectura de `main`

```
Navegador · React 19 / Vite / Firebase Auth
| REST /api/* · token Gmail cuando procede
▼
Express 5 · server.ts · 0.0.0.0:3000
├─ scraping SCPU con Cheerio
├─ extracción URL con Gemini 2.5 Flash
├─ envío y detección de respuestas con Gmail API
├─ cron 09:00 y 17:00 · Europe/Madrid
└─ lectura/escritura síncrona de data.json
```

El build ejecuta `vite build` y empaqueta `server.ts` con esbuild como `dist/server.cjs`. El repositorio no contiene Dockerfile, manifiesto Cloud Run, workflow CI ni definición de volumen persistente; Git no permite reproducir ni vincular el despliegue declarado con el SHA auditado.

Modelo de datos

Cada apartamento registra UUID, casero, superficie, capacidad, precio, ubicación, descripción, email, enlace, condiciones, contrato, estado y fecha. Puede añadir origen manual y datos de respuesta. El enlace funciona como clave de deduplicación.

Estados: `new`, `sent`, `following` y `discarded`.

Manual funcional de `main`

Descubrimiento automático

1. **Extraer Nuevos Pisos SCPU** o el cron inicia el proceso.
2. El servidor recorre hasta cinco páginas y limita el lote a 40 enlaces.
3. Cheerio extrae casero, email, precio, superficie, zona y contrato.
4. Se traducen expresiones croatas mediante sustituciones locales.
5. Se excluyen contratos detectados como anuales y se deduplica por URL.
6. El resultado se guarda y aparece en **Nuevas Oportunidades**.

La implementación contradice la afirmación documental “sin límite artificial”: hay un máximo de cinco páginas y 40 enlaces. Además, el scraper fija la capacidad como `Ver descripción`, aunque la regla de interfaz exige un número de personas.

Importación manual con IA

1. **Add URL**  abre el formulario.
2. El backend descarga la URL y elimina etiquetas ruidosas.
3. Envía hasta 12.000 caracteres a Gemini 2.5 Flash con esquema JSON.
4. Si Gemini no está disponible, aplica una heurística.
5. Marca la ficha como manual y deduplica por URL exacta.

No es segura para exposición pública hasta restringir protocolos, DNS/IP, redes privadas, redirecciones, tamaño y tiempo: hoy una petición anónima puede ordenar al servidor descargar una URL arbitraria, creando riesgo de SSRF.

Contacto y seguimiento Gmail

1. Firebase Google Sign-In entrega un access token conservado en memoria.
2. El backend construye un correo bilingüe inglés/croata y llama a Gmail API.
3. La comprobación revisa hasta 25 hilos y compara remitentes con caseros.
4. Una respuesta mueve un anuncio enviado a `following`.

Existe un defecto de integridad: el endpoint responde `success` y cambia el estado a `sent` incluso si falta un token utilizable o Gmail devuelve un error. “Enviado” no prueba que el correo haya salido.

Gestión visual

La SPA ofrece Nuevas Oportunidades, En Seguimiento, Enviados sin respuesta e Histórico, orden por fecha o precio y cambio manual de estado. Los botones de producto están presentes. `package.json` usa React 19, aunque la arquitectura escrita declara React 18.

Material ErasmusHomes excluido

El dossier de Bolonia, los mockups UNIBO, el backend FastAPI de auditoría de fianzas y los generadores ReportLab se excluyen expresamente de la arquitectura, API, puntuación y manual funcional de PULA. Solo se comprobó que su ubicación canónica actual es el repositorio independiente `ratienza/ErasmusHomes`; no se auditó su funcionamiento.

API REST observada

main

Método	Ruta	Función	Autenticación
GET	<code>/api/config</code>	Client ID OAuth	No
GET	<code>/api/apartments</code>	Lista completa	No
POST	<code>/api/apartments/scrape</code>	Scraping y escritura	No
POST	<code>/api/apartments/manual-url</code>	Descarga y analiza URL	No
POST	<code>/api/gmail/check-replies</code>	Consulta Gmail y actualiza	Bearer de Google
PUT	<code>/api/apartments/{id}/status</code>	Modifica estado	No
POST	<code>/api/apartments/{id}/send</code>	Envía y marca <code>sent</code>	Bearer opcional en la práctica

No se observan OpenAPI versionado, rate limiting, sesión de aplicación, autorización por usuario ni validación estructural centralizada.

Riesgos y pendientes reales

Prioridad alta

- Proteger lecturas y mutaciones con autenticación y autorización.
- Bloquear SSRF en la importación manual.
- Sacar `data.json` y los contactos del repositorio; tratar los emails como datos personales.
- No marcar `sent` tras error o ausencia de Gmail.
- Definir persistencia compatible con Cloud Run y operaciones concurrentes.

Prioridad media

- Usar bloqueo y reemplazo atómico en vez de escrituras síncronas directas.
- Restringir CORS, limitar cuerpos y cargas, añadir rate limiting y cabeceras.
- Validar HTTP del scraper, fijar timeouts y evitar logs OAuth detallados.
- Añadir tests unitarios, API, scraper con fixtures, Gmail simulado y CI.
- Elegir y fijar un único gestor de dependencias y lockfile.

Deriva documental

- Arquitectura: React 18; manifiesto: React 19.
- Documentación: scraping ilimitado; código: cinco páginas y 40 enlaces.
- `AGENTS.md` reserva Gmail y URL manual para v2.0; ambos ya están en `main`.
- La etiqueta y los artefactos ErasmusHomes están depositados en el historial de PULA pese a pertenecer al repositorio independiente `ratienza/ErasmusHomes`.

- Regla visual: prohíbe `Ver descripción` ; el scraper lo asigna a `size` .
- Cloud Run y Nginx declarados sin Dockerfile ni configuración reproducible.
- La automatización Python usa mock, OAuth local y `TEST_MODE = True` ; no pertenece al runtime Express demostrado.

Pruebas y límites

-  Checkout Nexus limpio y sincronizado.
-  Instalación temporal, `tsc --noEmit` y build Vite/esbuild.
-  No existe suite automatizada.
-  No se llamó a SCPU, Gemini, Gmail, Firebase ni datos vivos.
-  Cloud Run no fue inspeccionado ni validado.
-  ErasmusHomes solo se consultó para confirmar la separación; no fue auditado.

Operación futura y traslado

Antes de DigitalOcean se necesitan runtime reproducible, almacenamiento persistente, secretos fuera de Git, autenticación, backup/restore, healthcheck y tests. No deben reutilizarse datos o credenciales de Cloud Run implícitamente.

`Checkout ≠ Runtime` : PULA no se desplegó en Nexus. Cloud Run no se tocó.